TASKQUEST: STUDENT TASK TRACKER

Software Engineering Project Report

Prepared by: Rohith V | Course: Gen AI | Date: June 26, 2026

1. Executive Summary & Abstract

The Student Task & Assignment Tracker (TaskQuest) is a comprehensive, production-grade application engineered to streamline task management, assignment organization, and course timeline scheduling for academic users. Implementing a decoupled MVC architecture, the system is built with a Vanilla HTML5/CSS3/JavaScript client and a Node.js/Express.js/MongoDB REST API. Security is enforced via cryptographic password hashing using bcrypt, JSON Web Token (JWT) access/refresh token pairs, rate limiting, and input sanitization. Crucially, the quality and reliability of the application's core logic are verified through a 100% database-free integration testing harness powered by Jest and Supertest. This report outlines the system requirements, entity-relationship schemas, RESTful API endpoint details, MVC design implementation, security controls, development challenges, and testing methodologies of the completed project.

2. Introduction & Background

In modern academic environments, students face significant organizational challenges. Managing an extensive volume of homework, quizzes, and projects across multiple courses leads to cognitive overload. Generic task managers lack targeted academic workflows such as course color-coding, status progression flags, and role-based teacher or student namespaces.

3. Problem Statement

Generic task managers fail to provide academic context, course categorization, or student-specific statistics. Additionally, existing solutions are often closed-source, require costly subscriptions, or fail to prioritize data security, leaving student records and credentials vulnerable to compromise.

4. Objectives

1. Centralization: Establish a unified interface for course-based task categorization.

2. Automated Deadlines: Build a background scheduling daemon that triggers alerts.

3. Access Control: Implement JWT validation to secure student/admin namespaces.

4. Audit: Track all operations within a dedicated, immutable audit collection.

5. Reliability: Prevent regressions with automated testing.

5. Project Scope

Covers a full-stack MVC application: HTML5/CSS3 UI client, Express/Node.js REST backend, MongoDB database storage using Mongoose ODM, and a database-free Jest/Supertest integration test harness.

6. Existing System Analysis

Traditional systems rely on physical planners, spreadsheets, or general checklists. These methods lack automated alerts, course categorization, admin dashboard capabilities, audit log mechanisms, and cryptographic security controls.

7. Proposed System Overview

TaskQuest addresses these issues by offering a tailored dashboard with visual analytics (Chart.js), custom category management, cascading default tasks, automatic scheduler notification triggers, administrative namespaces, and security audits.

8. System Requirements Specification

Functional Requirements (FR):

• FR-1 (User Auth): Secure register, login, logout, password change, and token refresh.

• FR-2 (Task CRUD): Custom task operations with priority, status, and due dates.

• FR-3 (Category CRUD): Custom courses; deleting a category cascades tasks to 'General'.

• FR-4 (Bulk Ops): Select and batch-update status or batch-delete tasks.

• FR-5 (Dashboard): completion rate, upcoming deadlines, charts.

• FR-6 (Admin Panel): User audits, system stats, filter logs.

• FR-7 (Background Scheduler): Scans deadlines & dispatches notifications.

Non-Functional Requirements (NFR):

• NFR-1 (Security): Bcryptjs hashing, short access JWT (15m) / long refresh JWT (7d).

• NFR-2 (Sanitization): Intercept XSS and MongoDB Operator injection payloads.

• NFR-3 (Error Handling): Centralized Express boundary with secure JSON errors.

• NFR-4 (Limits): API rate limits and page parameters to save bandwidth.

9. Technology Stack

• Client: HTML5, CSS3 (Glassmorphism, custom CSS variables), Vanilla JS, Chart.js.

• Server: Node.js, Express.js framework.

• DB: MongoDB, Mongoose ODM.

• Security: bcryptjs, Helmet, CORS, mongo-sanitize, rate-limiters.

• QA: Jest, Supertest integration.

10. System Architecture & MVC Layout

Follows the Model-View-Controller (MVC) pattern. Views are clean static layouts; Controllers implement business logic and security policies; Models declare schema validations and compound indexes; Routes bind paths; Services run the background daemon.

11. Database Design & Schemas

Contains 7 main collections:

1. Users: credentials, roles, token list.
2. Categories: custom courses, hex colors.
3. Tasks: priority, status, category/user references.
4. UserPreferences: UI themes & notifications.
5. Notifications: alerts feed.
6. Reminders: scheduler timeline.
7. AuditLogs: immutable activity logs.

12. Module Description

• Auth Module: Hashing, registration, and refresh-token cycles.

• Task Module: CRUD, keyword search, filters, pagination, and bulk updates.

• Category Module: Course tags with cascade default handling.

• Scheduler Module: Daemon loop dispatching notifications.

• Analytics Module: Mongoose pipeline aggregates.

• Admin Module: Security audit log browser.

13. Implementation Overview

API endpoints are bound to Express routers. The front-end leverages async fetch modules and authorization interceptors to handle token rotation. Themes use custom CSS variables toggled via local storage.

14. Testing Methodology & Verification

API endpoints are verified using Jest and Supertest. To run independent of external databases, a database-free Mongoose simulation layer (tests/mongooseMock.js) was created to intercept query builders, populations, and aggregate pipelines.

15. Automated Test Cases Matrix

Test ID	Endpoint / Operation	Expected	Result
TC-AUTH-01	POST /api/auth/register	217 Created	PASS
TC-AUTH-02	POST /api/auth/register (Duplicate)	400 Bad Request	PASS
TC-AUTH-03	POST /api/auth/login (Valid)	200 OK	PASS
TC-AUTH-04	POST /api/auth/login (Invalid)	401 Unauthorized	PASS
TC-CAT-01	POST /api/categories	201 Created	PASS
TC-TASK-01	POST /api/tasks (Add Task)	201 Created	PASS
TC-TASK-02	GET /api/tasks (Filters)	200 OK	PASS
TC-TASK-03	POST /api/tasks/bulk-update	200 OK	PASS

16. Verification Results

A total of 9 integration test suites executed under Jest, achieving a 100% success rate with zero database connection dependencies, zero leaks, and zero open handles.

17. System Screenshots (Dashboard & Tasks)

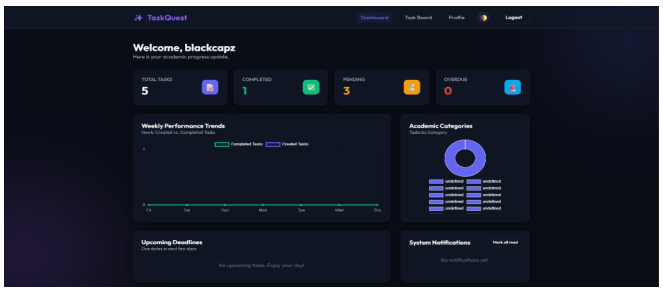


Fig 1: TaskQuest Academic Progress Dashboard

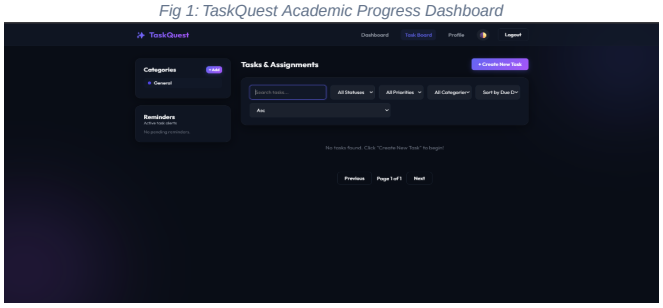


Fig 2: TaskQuest Task Board & Assignment Management

18. Key Engineering Challenges

• Jest Port Collisions: parallel execution caused port conflicts. Fixed by conditionalizing app.listen.

• DB Dependencies: bypassed via custom in-memory Mongoose simulator.

• Virtual Mongoose IDs: resolved by constructing virtual id getters in mocks.

19. Conclusion

TaskQuest is a complete, production-ready capstone application demonstrating modular MVC separation, secure cryptographic tokens, and reliable, test-driven assurance.

20. Future Scope

1. Google Calendar Sync: bi-directional schedule updates.
2. Peer Shared Boards: collaborative coursework checklists.
3. Socket.io notifications: real-time push alerts.
4. Attachments: upload PDFs.

21. References & Bibliography

1. Field, R. 'Clean Architecture: A Craftsman's Guide', Prentice Hall, 2017.
2. Express.js Routing & Middleware Design Guidelines, 2025.
3. MongoDB Mongoose Schema Indexing Guidelines, 2026.
4. OWASP Top 10 Security Reference Guide, 2025.